Beam Pulse Subsystem — Comprehensive Test Plan

Subsystem: `BeamPulseSubsystem` (`subsystem/beam_pulse/beam_pulse.py`)
Driver: `BCONDriver` (`instrumentctl/BCON/bcon_driver.py`)
Protocol: Modbus RTU over serial (Arduino Mega firmware)
Date: 2026-02-26

Table of Contents

1. Test Environment & Prerequisites
2. Unit Tests — Initialization
3. Unit Tests — Input Validation
4. Unit Tests — Armed State Management
5. Unit Tests — CSV Sequence Parsing
6. Unit Tests — Public API (No Hardware)
7. BCONDriver — Connection Management
8. BCONDriver — Register I/O
9. BCONDriver — Channel Control (Hardware-in-the-Loop)
10. BCONDriver — Safety & Fault Management
11. BCONDriver — Background Polling Thread
12. Integration Tests — Subsystem ↔ Driver
13. GUI Tests — Status Bar & Connection Indicator
14. GUI Tests — Manual Control Tab
15. GUI Tests — CSV Sequence Tab
16. GUI Tests — Tab Switching & Panel Visibility
17. GUI Tests — Sync Start / Sync Stop
18. Safety & Interlock Tests
19. Regression Tests — Edge Cases & Robustness
20. Performance & Timing Tests

---

1. Test Environment & Prerequisites

- ☐ Python ≥ 3.9 installed with `pymodbus`, `pyserial`, and `tkinter` available
- ☐ Virtual environment activated (`source .venv/bin/activate`)
- ☐ BCON firmware flashed on Arduino Mega and accessible via serial port
- ☐ Serial port confirmed with `python -m serial.tools.list_ports`
- ☐ Hardware interlock line is in a safe (permissive) state before running hardware tests
- ☐ `pytest` installed (`pip install pytest pytest-mock`)
- ☐ `docs/` directory created for test artefacts
- ☐ `sequences/` directory exists (auto-created by subsystem, but confirm)
- ☐ `presets/` directory exists (auto-created by subsystem, but confirm)

---

2. Unit Tests — Initialization

2.1 No-port, no-GUI instantiation

- [ ] `BeamPulseSubsystem()` with no arguments creates object without raising
- [ ] `bcon_driver` is `None` when `port=None`
- [ ] `bcon_connection_status` initializes to `False`
- [ ] `beams_armed_status` initializes to `False`
- [ ] `beam_on_status` initializes to `[False, False, False]`
- [ ] `_active_channels` initializes to an empty `set`
- [ ] `_seq_steps` initializes to an empty `list`
- [ ] `pulsing_behavior` is a plain string `"DC"` (not a `tk.StringVar`) when no parent frame
- [ ] `beam_a_duration`, `beam_b_duration`, `beam_c_duration` are plain `float` values when no parent frame

2.2 Port-only instantiation (no GUI)

- [ ] `BeamPulseSubsystem(port="/dev/ttyUSB0")` creates a `BCONDriver` instance
- [ ] Background auto-connect thread is started (daemon thread visible in `threading.enumerate()`)
- [ ] `_ui_queue` is wired to `bcon_driver` via `set_ui_queue`

2.3 Directory creation

- [ ] `sequences/` directory is created if it does not exist
- [ ] `presets/` directory is created if it does not exist
- [ ] Pre-existing directories are not overwritten or cleared

---

3. Unit Tests — Input Validation (`_validate_and_get_config`)

3.1 OFF mode

- [ ] Returns `{'mode': 'OFF', 'duration_ms': 0, 'count': 1}` regardless of duration/count widget content
- [ ] Does **not** show an error messagebox

3.2 DC mode

- [ ] Returns `{'mode': 'DC', 'duration_ms': 0, 'count': 1}` regardless of duration/count widget content
- [ ] Does **not** show an error messagebox

3.3 PULSE mode — valid input

- [ ] Duration `100` → returns `{'mode': 'PULSE', 'duration_ms': 100, 'count': 1}`
- [ ] Count in widget is ignored; returned count is always forced to `1`
- [ ] Duration `1` (minimum non-zero) is accepted

3.4 PULSE mode — invalid input

- [ ] Duration `0` → shows "Invalid Configuration" messagebox, returns `None`
- [ ] Duration empty (`""`) → shows messagebox, returns `None`

- ☐ Duration non-numeric (e.g., `"abc"`) → shows messagebox, returns `None`
- ☐ Duration negative value → shows messagebox, returns `None`

3.5 PULSE_TRAIN mode — valid input

- ☐ Duration `50`, count `2` → returns `{'mode': 'PULSE_TRAIN', 'duration_ms': 50, 'count': 2}`
- ☐ Duration `50`, count `10` is accepted
- ☐ Duration `60000` (upper boundary reasonable value) is accepted

3.6 PULSE_TRAIN mode — invalid input

- ☐ Count `< 2` (e.g., `1`) → shows messagebox mentioning "≥ 2", returns `None`
- ☐ Count `0` → shows messagebox, returns `None`
- ☐ Count empty → shows messagebox, returns `None`
- ☐ Duration `0` with valid count → shows messagebox, returns `None`

3.7 Unknown mode string

- ☐ Mode label not in `MODE_LABEL_TO_CODE` → shows messagebox, returns `None`

3.8 Channel index out of range (no channel_vars entry)

- ☐ `_validate_and_get_config(ch=5)` falls back to `get_channel_config(5)` which returns defaults

---

4. Unit Tests — Armed State Management

4.1 `_require_armed` guard

- ☐ Returns `False` and logs warning when `beams_armed_status` is `False`
- ☐ Returns `True` when `beams_armed_status` is `True`

4.2 `arm_beams()`

- ☐ Sets `beams_armed_status = True`
- ☐ Calls `bcon_driver.arm()` when driver is present
- ☐ Returns `True` when driver is `None` (still sets armed flag)
- ☐ Calls `_update_armed_button_states(True)`

4.3 `disarm_beams()`

- ☐ Sets `beams_armed_status = False`
- ☐ Calls `bcon_driver.stop_all()` when driver is present
- ☐ Sets all `beam_on_status` entries to `False` via `set_all_beams_status(False)`
- ☐ Calls `_update_armed_button_states(False)`
- ☐ Returns `True`

4.4 `_update_armed_button_states`

- ☐ All buttons in `_armed_gated_buttons` set to `"normal"` when `armed=True`
- ☐ All buttons in `_armed_gated_buttons` set to `"disabled"` when `armed=False`
- ☐ `seq_run_btn` enabled only when `armed=True` **and** `_seq_steps` is non-empty
- ☐ `seq_run_btn` stays disabled when `armed=True` but `_seq_steps` is empty
- ☐ Stop buttons (`sync_stop_btn`, `seq_stop_btn`) are **never** modified by this method

---

5. Unit Tests — CSV Sequence Parsing (`_load_sequence`)

> Tests use pre-written CSV files placed in `sequences/test_fixtures/`

5.1 Valid sequence — basic structure

- ☐ Lines starting with `#` are skipped
- ☐ Header line starting with `step` (case-insensitive) is skipped
- ☐ Steps sharing the same step number are grouped into one tuple
- ☐ Resulting `_seq_steps` list is sorted by step number
- ☐ `seq_file_lbl` displays filename and step count

5.2 Channel `ALL` expansion

- ☐ Row with `ch=ALL` creates three sub-rows (ch=0, ch=1, ch=2)

5.3 Mode parsing

- ☐ `OFF`, `DC`, `PULSE`, `PULSE_TRAIN` all recognized (case-insensitive)
- ☐ Unknown mode string raises `ValueError` and shows error messagebox
- ☐ `PULSE_TRAIN` with `count < 2` raises `ValueError` and shows error messagebox

5.4 Missing optional fields

- ☐ Missing `duration_ms` defaults to `100`
- ☐ Missing `count` defaults to `1`
- ☐ Missing `dwell_ms` defaults to `0`

5.5 Dwell time

- ☐ `dwell_ms` is taken from the **last** row of each step group
- ☐ Zero `dwell_ms` does not introduce an observable delay

5.6 Lines with fewer than 3 fields

- ☐ Rows with `< 3` comma-separated parts are silently skipped

5.7 Preview text widget

- ☐ After loading, `seq_preview_text` contains one line per channel-row
- ☐ Format: `Step N: CHX MODE dur=Yms cnt=Z dwell=Wms`

5.8 Run Sequence button state after load

- ☐ `seq_run_btn` enabled if `beams_armed_status` is `True`
- ☐ `seq_run_btn` remains disabled if `beams_armed_status` is `False`

5.9 Template save (`_save_sequence_template`)

- ☐ Written file contains the header comment block
- ☐ Written file contains the `step,ch,mode,duration_ms,count,dwell_ms` column header
- ☐ Written file parses successfully through `_load_sequence` without errors

---

6. Unit Tests — Public API (No Hardware)

6.1 `get_beam_status` / `set_beam_status`

- ☐ `set_beam_status(0, True)` → `get_beam_status(0)` returns `True`
- ☐ `set_beam_status(1, False)` → `get_beam_status(1)` returns `False`
- ☐ Index out-of-range (e.g., `3`) returns `False`

6.2 `set_all_beams_status`

- ☐ `set_all_beams_status(True)` sets all three `beam_on_status` to `True`
- ☐ `set_all_beams_status(False)` sets all three `beam_on_status` to `False`

6.3 `get_pulsing_behavior`

- ☐ Returns string value `"DC"` when no GUI is active
- ☐ Returns `.get()` of `tk.StringVar` when GUI is active

6.4 `get_beam_duration`

- ☐ Returns `50.0` default for each index in headless mode
- ☐ Index `0` → `beam_a_duration`, `1` → `beam_b_duration`, `2` → `beam_c_duration`
- ☐ Out-of-range index returns `50.0`

6.5 `get_channel_config`

- ☐ Returns `{'mode': 'PULSE', 'duration_ms': 100, 'count': 1}` defaults when no GUI/channel_vars
- ☐ Returns widget values when channel_vars are populated

6.6 `get_deflect_beam_status`

- ☐ Returns `True` when **any** `beam_on_status` is `True`
- ☐ Returns `False` when all are `False`

6.7 `set_channel_status_callback` / `set_channel_enable_getter`

- ☐ Callback registered via `set_channel_status_callback(fn)` is stored in `_channel_status_callback`
- ☐ Getter registered via `set_channel_enable_getter(fn)` is stored in `_ch_enable_getter`

6.8 `safe_shutdown`

- ☐ Calls `disarm_beams()` (sets armed=False, stops all channels)
- ☐ Returns `True`

6.9 `get_integration_status`

- ☐ Returns dict with keys `has_dashboard_callback` and `bcon_connected`
- ☐ `has_dashboard_callback` is `False` before registering callback
- ☐ `has_dashboard_callback` is `True` after `set_dashboard_beam_callback(fn)`

---

7. BCONDriver — Connection Management

7.1 Successful connection

- ☐ `connect()` returns `True` when serial port is available and firmware responds
- ☐ `is_connected()` returns `True` after successful connect
- ☐ Background poll thread is started after connect
- ☐ UI queue receives `("connected", True)` message
- ☐ `SETTLE_TIME` (2.5 s) elapses before first Modbus frame (confirm with serial sniffer or timing log)

7.2 Failed connection — bad port

- ☐ `connect()` returns `False` for a non-existent port
- ☐ `is_connected()` returns `False`
- ☐ UI queue receives `("connected", False)` message
- ☐ No poll thread is left running

7.3 Failed connection — pymodbus not installed

- ☐ `connect()` returns `False` and logs an `ERROR` when `ModbusClient is None`

7.4 Disconnect

- ☐ `disconnect()` sets `is_connected()` to `False`
- ☐ Poll thread is joined within 3 s
- ☐ UI queue receives `("connected", False)` message
- ☐ Subsequent `enqueue_write` calls are safe (no exception)

7.5 Re-connect after disconnect

- ☐ Calling `connect()` again after `disconnect()` re-opens the port successfully
- ☐ Poll thread is restarted cleanly

7.6 `ping()`

- ☐ Returns `True` when connected and firmware responds to register read
- ☐ Returns `False` when not connected

7.7 Arduino DTR reset settle time

- ☐ Custom `settle_s=0` skips the wait (useful for tests with a stable device)
- ☐ Default `settle_s=None` uses `SETTLE_TIME = 2.5`

---

8. BCONDriver — Register I/O

8.1 `enqueue_write` + poll thread processing

- ☐ Written value appears in the firmware register within one poll cycle
- ☐ Write order is preserved (FIFO queue)
- ☐ Multiple writes in quick succession are all delivered

8.2 `write_register_immediate`

- ☐ Returns `True` and completes synchronously when connected
- ☐ Returns `False` when not connected

8.3 `get_register` / `get_registers`

- ☐ Returns `0` for all registers at startup before first poll
- ☐ Returns live values after first successful poll
- ☐ `get_registers()` returns a **copy** (mutating it does not affect the cache)

8.4 Pymodbus version compatibility

- ☐ Works with pymodbus v3.x API (`framer="rtu"`, `device_id=`)
- ☐ Falls back to older API (`method="rtu"`, `unit=`) without raising

8.5 Register map boundaries

- ☐ Read block `[0, 34)` covers control + all three channel parameter blocks
- ☐ Read block `[100, 106)` covers system status registers
- ☐ Read blocks `[110, 119)`, `[120, 129)`, `[130, 139)` cover channel status registers
- ☐ `TOTAL_REGS = 160` is sufficient to index all defined registers

---

9. BCONDriver — Channel Control (Hardware-in-the-Loop)

> All tests in this section require the BCON hardware connected.

9.1 Channel OFF

- ☐ `set_channel_off(1)` → CH1 status register mode = `OFF` (0) within one poll cycle
- ☐ Out-of-range channel `0` or `4` logs an error without sending a Modbus frame

9.2 Channel DC

- ☐ `set_channel_dc(2)` → CH2 status mode = `DC` (1)
- ☐ Output level register (`+8`) = `1` while in DC mode

9.3 Channel PULSE

- ☐ `set_channel_pulse(1, 100)` → CH1 mode = `PULSE` (2), remaining counts down to 0
- ☐ `set_channel_pulse(1, 100, 1)` fires exactly one pulse
- ☐ Duration `0` or negative logs error, does **not** enqueue write
- ☐ Duration `> 60000` logs error, does **not** enqueue write

9.4 Channel PULSE_TRAIN

- ☐ `set_channel_pulse_train(3, 50, 5)` → CH3 mode = `PULSE_TRAIN` (3), remaining decrements
- ☐ Count `< 2` logs error, does **not** enqueue write
- ☐ Remaining value reaches `0` after all pulses complete

9.5 `set_channel_params` (write params without mode change)

- ☐ Writes `duration_ms` and `count` registers without altering mode register
- ☐ `duration_ms=0` does not write the duration register
- ☐ `count=0` does not write the count register

9.6 `toggle_channel_enable`

- ☐ Writing to `CH_ENABLE_TOGGLE_OFF` register toggles enable status bi-directionally
- ☐ `is_channel_enabled(ch)` reflects updated state after one poll cycle

9.7 `stop_all`

- ☐ Sets all three channel mode registers to `OFF` (0) immediately
- ☐ All channels become inactive within one poll cycle

9.8 `set_channel_mode` (generic)

- ☐ All four mode strings `"OFF"`, `"DC"`, `"PULSE"`, `"PULSE_TRAIN"` dispatch correctly
- ☐ Unknown mode string logs error without sending Modbus frame

---

10. BCONDriver — Safety & Fault Management

10.1 ARM / CLEAR_FAULT

- ☐ `arm()` writes value `3` to `REG_COMMAND` (register 2)
- ☐ System state transitions from `FAULT_LATCHED` → `READY` after ARM command
- ☐ `clear_fault()` is an alias; identical behaviour

10.2 Interlock status

- ☐ `is_interlock_ok()` returns `True` when hardware interlock is satisfied
- ☐ `is_interlock_ok()` returns `False` when interlock line is de-asserted
- ☐ Safety label in GUI updates to "Interlock: locked" when interlock fails

10.3 Watchdog

- ☐ `set_watchdog(2000)` writes 2000 to `REG_WATCHDOG_MS` (register 0)

- ☐ After watchdog timeout with no Modbus activity, firmware enters `SAFE_WATCHDOG` state
- ☐ `is_watchdog_ok()` returns `False` in `SAFE_WATCHDOG` state
- ☐ Resuming Modbus writes and sending ARM clears the watchdog state

10.4 Latched fault

- ☐ `is_fault_latched()` returns `True` when `REG_FAULT_LATCHED` is non-zero
- ☐ GUI safety label shows `FAULT` suffix when fault is latched
- ☐ Fault clears after `arm()` and interlock is satisfied

10.5 Overcurrent detection

- ☐ `is_channel_overcurrent(ch)` returns `True` when overcurrent bit is set in firmware
- ☐ GUI overcurrent canvas turns red when overcurrent is active

10.6 System state codes

- ☐ State `0` → `"READY"`
- ☐ State `1` → `"SAFE_INTERLOCK"`
- ☐ State `2` → `"SAFE_WATCHDOG"`
- ☐ State `3` → `"FAULT_LATCHED"`
- ☐ Unknown state code → `"UNKNOWN"`

---

11. BCONDriver — Background Polling Thread

11.1 Poll interval

- ☐ Registers are polled approximately every `POLL_INTERVAL = 0.3` s
- ☐ UI queue receives `("regs", <list>)` message only when registers **change**

11.2 Consecutive poll failure handling

- ☐ Each failed read increments `_poll_errors`
- ☐ After `MAX_POLL_ERRORS = 4` consecutive failures, `_auto_disconnect()` is called
- ☐ After auto-disconnect, `is_connected()` returns `False`
- ☐ UI queue receives `("connected", False)` after auto-disconnect

11.3 Poll thread lifecycle

- ☐ Thread is a daemon thread (does not block process exit)
- ☐ `_stop_poll_thread()` joins within 3 s
- ☐ Thread does not restart automatically after auto-disconnect (requires explicit `connect()`)

11.4 Write queue & poll interleaving

- ☐ Queued writes are processed **before** each new poll cycle
- ☐ No write is dropped when multiple writes are enqueued in rapid succession

11.5 UI queue messages

- ☐ `("wrote", reg, val)` sent after each successful register write
- ☐ `("error", msg)` sent on write failure
- ☐ `("error", msg)` sent on read failure (deduplicated: same message not repeated)

---

12. Integration Tests — Subsystem ↔ Driver

12.1 Auto-connect on startup

- ☐ `BeamPulseSubsystem(port=<port>)` triggers background auto-connect thread
- ☐ After successful connect, UI queue receives `("connected", True)`
- ☐ After failed connect, `_log_event` records failure message

12.2 `send_channel_config` (single channel beam on)

- ☐ Requires `beams_armed_status = True`; returns `False` otherwise
- ☐ Validates via `_validate_and_get_config`; returns `False` on bad input
- ☐ Enqueues correct mode + params writes to `bcon_driver`
- ☐ Sets `beam_on_status[ch] = True`
- ☐ Invokes `_dashboard_beam_callback(ch, True)` if registered

12.3 `send_channel_off` (single channel beam off)

- ☐ Does **not** require armed state
- ☐ Enqueues `BCONMode.OFF` to the channel mode register
- ☐ Sets `beam_on_status[ch] = False`
- ☐ Invokes `_dashboard_beam_callback(ch, False)` if registered

12.4 `_update_ui_from_registers` (register → widget sync)

- ☐ Channel status label updated with mode string and output level
- ☐ "Remaining" label updated with remaining pulse count
- ☐ DC mode treated as "running" (`is_running=True`) regardless of remaining count
- ☐ Non-zero remaining (PULSE/PULSE_TRAIN active) triggers `_set_manual_channel_lock(ch, True)`
- ☐ Channel with remaining=0 and mode=OFF triggers `_set_manual_channel_lock(ch, False)`
- ☐ `_channel_status_callback(ch, mode_code, remaining)` invoked for each channel

12.5 `_safe_fill` utility

- ☐ Overwrites entry widget only when current value is `""` or `"0"`
- ☐ Does **not** overwrite a non-zero user-entered value
- ☐ Silently skips when widget is in `"disabled"` state

12.6 Channel enable getter integration

- ☐ `_sync_start` calls `_ch_enable_getter()` when registered
- ☐ Channels with `enable_state=False` are skipped and logged
- ☐ Defaults to all channels enabled when no getter is registered

12.7 `connect()` / `disconnect()` / `is_connected()` delegation

- ☐ `BeamPulseSubsystem.connect()` delegates to `BCONDriver.connect()`
- ☐ On success, calls `bcon_driver.set_watchdog(2000)` and `set_telemetry(500)`
- ☐ `BeamPulseSubsystem.disconnect()` calls `stop_all()`, waits 0.3 s, then disconnects driver

---

13. GUI Tests — Status Bar & Connection Indicator

- ☐ BCON indicator canvas is **red** on startup (disconnected)
- ☐ BCON indicator canvas turns **green** after successful connect
- ☐ BCON indicator canvas returns to **red** after disconnect or auto-disconnect
- ☐ "Connect" button relabels to "Disconnect" when connected
- ☐ "Disconnect" button relabels to "Reconnect" when disconnected
- ☐ Safety label shows `"Interlock: ok | Watchdog: ok"` in normal operation
- ☐ Safety label shows `"Interlock: locked"` when interlock de-asserted
- ☐ Safety label shows `"Watchdog: expired"` after watchdog timeout
- ☐ Safety label shows `"FAULT"` suffix when fault is latched
- ☐ Watchdog entry initializes to `"2000"` ms
- ☐ "Set" button writes watchdog value to BCON
- ☐ Log label updates with the most recent event string
- ☐ `_manual_connect` button triggers background re-connect thread
- ☐ Connect button is disabled while re-connect is in progress

---

14. GUI Tests — Manual Control Tab

14.1 Channel card layout

- ☐ Three channel cards appear side-by-side
- ☐ Each card has a mode combobox, duration entry, count entry, status label, remaining label

14.2 Mode combobox — widget state transitions

- ☐ Selecting `"OFF"` → duration and count entries become `"disabled"`
- ☐ Selecting `"DC"` → duration and count entries become `"disabled"`
- ☐ Selecting `"PULSE"` → duration entry enabled, count entry disabled (forced to `"1"`)
- ☐ Selecting `"PULSE_TRAIN"` → both duration and count entries enabled
- ☐ Default mode is `"PULSE"` at startup (count grayed out by default)

14.3 Duration entry — digits-only validation

- ☐ Typing a digit is accepted
- ☐ Typing a letter is rejected (entry unchanged)
- ☐ Typing a decimal point is rejected
- ☐ Typing a negative sign is rejected
- ☐ Empty field is accepted (validation doesn't block clearing)

14.4 Channel lock during active pulse

- ☐ While a channel is running (remaining > 0 or mode=DC), mode/duration/count widgets are locked (`"disabled"`)
- ☐ When the channel returns to OFF with remaining=0, widgets are unlocked to `"readonly"` / `"normal"` correctly

14.5 Status and remaining labels

- ☐ Status label updates from register poll to show live mode string and output level
- ☐ Remaining label shows `"Remaining: N"` count from register poll

14.6 `_safe_fill` — auto-fill from register cache

- ☐ Duration entry auto-fills from `pulse_ms` register when current value is `""` or `"0"`
- ☐ Count entry auto-fills from `count` register when current value is `""` or `"0"`
- ☐ Non-zero user values are **not** overwritten

---

15. GUI Tests — CSV Sequence Tab

- ☐ Preview text initially empty, state `"disabled"`
- ☐ `seq_file_lbl` starts with `"No sequence loaded"` in gray
- ☐ `seq_progress_lbl` starts empty
- ☐ After loading a valid CSV, `seq_file_lbl` shows filename and step count
- ☐ After loading, preview text is populated and read-only
- ☐ "Load CSV" button opens a file dialog filtered to `*.csv`
- ☐ "Save Template" button opens a save dialog and writes a valid template file
- ☐ "Run Sequence" button is disabled before a sequence is loaded
- ☐ "Run Sequence" button is disabled even after loading if beams are not armed
- ☐ "Run Sequence" button becomes enabled after loading **and** arming
- ☐ "Stop Sequence" button is always accessible (not gated by armed state)
- ☐ Progress label updates to `"Step N/T (#stepnum)"` during playback
- ☐ Progress label shows `"Sequence complete"` after all steps finish
- ☐ Progress label shows `"Sequence stopped"` when manually stopped mid-run
- ☐ "Run Sequence" re-enables after sequence completes
- ☐ Calling "Stop Sequence" during active playback stops the background thread

---

16. GUI Tests — Tab Switching & Panel Visibility

- ☐ At startup (Tab 0 = Manual Control selected), Beam ON/OFF frame is visible
- ☐ At startup, Sync Start/Stop row is visible
- ☐ At startup, CSV buttons frame is **hidden**
- ☐ Switching to Tab 1 (CSV Sequence) hides Beam ON/OFF frame
- ☐ Switching to Tab 1 hides Sync Start/Stop row
- ☐ Switching to Tab 1 shows CSV buttons frame
- ☐ Switching back to Tab 0 restores original visibility state
- ☐ CH Enable/Disable row is **always visible** regardless of active tab
- ☐ Tab-switch binding is registered via `<<NotebookTabChanged>>` event

---

17. GUI Tests — Sync Start / Sync Stop

17.1 Sync Start button

- ☐ Disabled at startup (not armed)
- ☐ Enabled after `arm_beams()`
- ☐ Disabled again after `disarm_beams()`
- ☐ Clicking while not armed shows no effect (button is disabled, can't click)
- ☐ Clicking while armed collects configs from Manual Control tab for all enabled channels
- ☐ Enqueues phase-1 writes (duration + count) then phase-2 writes (mode) to driver
- ☐ Channels reported as disabled by `_ch_enable_getter` are skipped

17.2 Sync Stop button

- ☐ Always enabled (safety action)
- ☐ Clicking calls `bcon_driver.stop_all()`
- ☐ All channels return to OFF mode

17.3 Config collection for sync start

- ☐ OFF and DC channels: no duration/count writes (params not required)
- ☐ PULSE channels: writes duration, count=1, then mode
- ☐ PULSE_TRAIN channels: writes duration, count, then mode

---

18. Safety & Interlock Tests

18.1 Action blocking when not armed

- ☐ `_manual_apply` blocked (logs "Action blocked: beams are not armed")
- ☐ `_manual_set_mode` blocked
- ☐ `_manual_toggle_enable` blocked
- ☐ `_sync_write_params` blocked
- ☐ `_sync_start` blocked
- ☐ `_run_sequence` blocked
- ☐ `send_channel_config` returns `False`
- ☐ `set_channel_mode` (public API) returns `False`
- ☐ Stop actions (`_sync_stop_all`, `send_channel_off`, `stop_all_channels`) are **not** blocked

18.2 Arm / Disarm workflow

- ☐ Full workflow: instantiate → connect → arm → fire pulse → disarm → all channels off
- ☐ Disarm immediately stops any active PULSE_TRAIN
- ☐ After disarm, attempting to arm again restores full functionality

18.3 Safe shutdown

- ☐ `safe_shutdown("test reason")` leaves all channels in OFF mode
- ☐ `beams_armed_status` is `False` after safe shutdown
- ☐ All `beam_on_status` entries are `False`

18.4 Watchdog safety

- ☐ If the master process is killed, the firmware enters `SAFE_WATCHDOG` state within `watchdog_ms`
- ☐ Channels stop pulsing when watchdog expires on the firmware side
- ☐ Reconnecting and sending ARM clears the watchdog and resumes operation

18.5 Hardware interlock integration

- ☐ When interlock line drops, GUI shows "Interlock: locked"
- ☐ Action buttons remain enabled (the subsystem does not gate on software interlock state)
- ☐ Restoring the interlock + sending ARM allows pulsing to resume

---

19. Regression Tests — Edge Cases & Robustness

19.1 No driver (port=None) paths

- ☐ `_manual_apply` → returns early with a log warning, no crash
- ☐ `_sync_start` → returns early
- ☐ `_sync_stop_all` → no crash (driver is None, guard present)
- ☐ `connect()` → returns `False`
- ☐ `disconnect()` → no crash
- ☐ `stop_all_channels()` → returns `False`
- ☐ `ping()` → returns `False`

19.2 Sequence player edge cases

- ☐ Empty CSV file (only comments/header) → `_seq_steps = []`, no error
- ☐ Single-step sequence plays and exits cleanly
- ☐ Sequence with ALL channel rows correctly expands to 3 channels per step
- ☐ `_stop_sequence()` while sequence is idle (thread not running) → no exception
- ☐ Starting a sequence while one is already running is ignored (thread alive check)
- ☐ Sequence with `dwell_ms=0` does not hang

19.3 Thread safety

- ☐ `_log()` called from background thread routes message via `parent_frame.after(0, ...)` (no direct tk call)
- ☐ `_ui_queue` is a `queue.Queue` and remains thread-safe under concurrent put/get
- ☐ `_regs_lock` protects `_regs` list from race conditions between poll thread and UI readers

19.4 Callback robustness

- ☐ Exception in `_dashboard_beam_callback` is caught and does not propagate
- ☐ Exception in `_channel_status_callback` is caught and does not propagate
- ☐ Exception in `_ch_enable_getter` falls back to `[True, True, True]`

19.5 Widget absence guards

- ☐ `update_bcon_connection_status()` is safe when `bcon_connection_canvas` does not exist

- ☐ `update_pulser_status_display()` is safe when canvases list is empty
- ☐ `_update_armed_button_states()` is safe when `_armed_gated_buttons` does not exist

19.6 Pulser status monitoring — no hardware

- ☐ `get_pulser_overcurrent_status()` returns `False` when driver is None
- ☐ `update_pulser_status_display()` does not crash when BCON is disconnected

---

20. Performance & Timing Tests

- ☐ Register poll latency: average round-trip Modbus read ≤ 50 ms at 115200 baud
- ☐ UI update latency: register change to GUI widget update ≤ 400 ms (two 200 ms ticks)
- ☐ Write-to-hardware latency: `enqueue_write` to firmware acknowledgement ≤ 350 ms (one poll interval + write processing)
- ☐ Sync Start jitter: time between first and last channel mode write ≤ 5 ms (measured with serial sniffer)
- ☐ Sequence step transition: dwell_ms accuracy within ±100 ms for dwell ≥ 500 ms
- ☐ Connection settle time: 2.5 s delay after port open before first Modbus frame
- ☐ Auto-disconnect trigger: occurs within `MAX_POLL_ERRORS × POLL_INTERVAL = 4 × 0.3 = 1.2 s` of communication failure
- ☐ Poll thread CPU usage: ≤ 2 % of a single core during steady-state polling

---

Appendix A — Register Quick Reference

Register	Symbol	Description
0	`REG_WATCHDOG_MS`	Watchdog timeout (ms) – write 0 to disable
1	`REG_TELEMETRY_MS`	Telemetry/poll interval (ms)
2	`REG_COMMAND`	Command: 0=NOP, 3=ARM/CLEAR_FAULT
10, 20, 30	`CH_BASE[0-2]`	Channel 1/2/3 parameter base addresses
+0	`CH_MODE_OFF`	Requested mode (0=OFF,1=DC,2=PULSE,3=PT)
+1	`CH_PULSE_MS_OFF`	Pulse duration (ms)
+2	`CH_COUNT_OFF`	Pulse count
+3	`CH_ENABLE_TOGGLE_OFF`	Write 1 to toggle enable
100	`REG_SYS_STATE`	System state code
102	`REG_FAULT_LATCHED`	Latched fault flag
103	`REG_INTERLOCK_OK`	Interlock OK flag
104	`REG_WATCHDOG_OK`	Watchdog OK flag
110, 120, 130	`REG_CH_STATUS_BASE + n×10`	Channel 1/2/3 status base
+0	(status)	Actual mode

Register	Symbol	Description
+3	(status)	Remaining pulses
+4	(status)	Enable status
+6	(status)	Overcurrent flag
+8	(status)	Output level

Appendix B — Mode Encoding

Label	Code	Duration used	Count used
`OFF`	0	No	No
`DC`	1	No	No
`PULSE`	2	Yes	Always 1
`PULSE_TRAIN`	3	Yes	Yes (≥ 2)

Appendix C — CSV Sequence Format

```
# Comment lines are ignored
step,ch,mode,duration_ms,count,dwell_ms
1,1,PULSE,100,1,500
1,2,DC,,,500
2,ALL,OFF,,,1000
3,1,PULSE_TRAIN,50,5,0
```

- Rows sharing a `step` number are launched simultaneously (same sync group)
- `ch` can be `1`, `2`, `3`, or `ALL`
- `dwell_ms` is taken from the last row of each step group
- `duration_ms` and `count` default to `100` and `1` respectively when omitted